

Chap 6 (Graph sans circuits)

Fonction de range

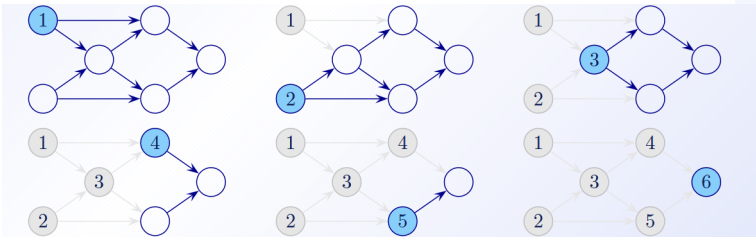
C'est une fonction qui permette de numérotée les sommet du graphe (tri topologique)

Un graph sans cercuit est un graph qui admet une fonction de rang

Tri topologic

Tant qu'il reste des sommets non numérotés faire

- Identifier un sommet sans prédécesseurs
- Le numéroté (à la suite de celui de l'itération précédente)
- Le supprimer du graphe



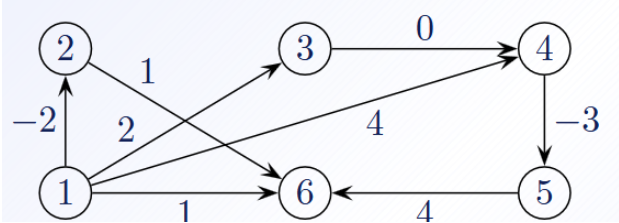
Complexité : O(N+M)

Théorème. Les dates de fin de traitement obtenues lors de l'exploration en profondeur d'un graphe sans circuits G définissent une fonction de rang pour le graphe transposé G^T .

Chemin le plus court

Vue qu'on n'a pas de cercuit, on peut just appliquer Bellman, faut juste traiter les sommet par

numérotation topologic



Somet traiter	1	2	3	4	5	6
1	-(0)	----	----	----	----	----
2	-(0)	1(-2)	----	----	----	----
3	-(0)	1(-2)	1(2)	----	----	----
4	-(0)	1(-2)	1(2)	3(2)	----	----
5	-(0)	1(-2)	1(2)	3(2)	4(-1)	----
6	-(0)	1(-2)	1(2)	3(2)	4(-1)	2(-1)

pred (distance)

Algorithme PCC_SANSCIRCUITS

- Tri topologic de $G \rightarrow G'$
- BellMan sur G' (source sommet #1)

Complexité : O(N+M)

Les graphes potentiels-ÉTAPES

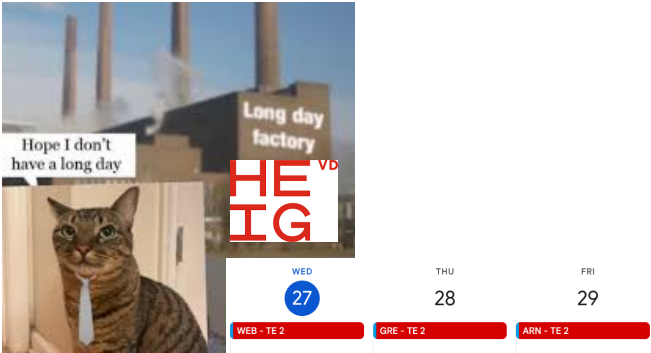
les sommets du graphe représentent différentes étapes de réalisation du projet global et les arcs correspondent aux tâches

Les graphes potentiels-TÂCHES

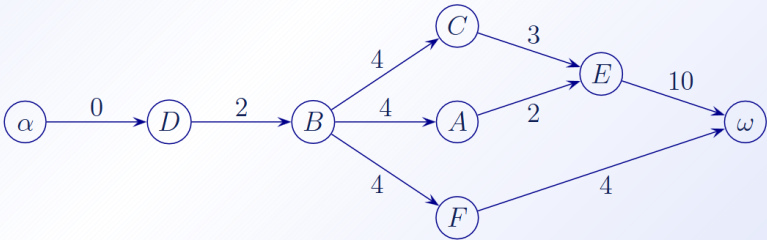
sommet du graphe correspond à une tâche et les arcs modélisent les contraintes d'antériorité

Pour le modélisé, c'est trivial

Faut just pas oublier de rajouter un sommet *début* d'où toutes les taches parte et un sommet *fin* qui représent la fin du projet et qui n'a donc pas de successeur



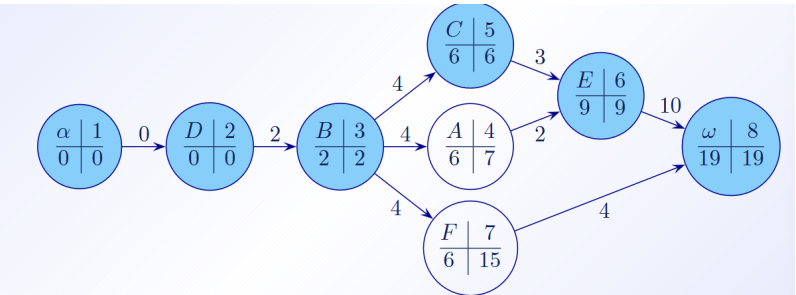
Tâche	Description	Durée (sem.)	Antériorités
D	Choix des stations	2	—
B	Accord administratif	4	D
C	Commande des décodeurs	3	B
A	Installation des antennes	2	B
E	Installation des décodeurs	10	C,A
F	Modification de la facturation	4	B



La date de **debut de réalisation** d'une tache est donc donner par son chemin le **plus long**

Chemin critique

Un chemin critique est un plus long chemin du sommet α (début des travaux) au sommet ω (fin des travaux) (il est donc composé uniquement de tâches critiques). La longueur D d'un chemin critique correspond à la durée minimale nécessaire à la réalisation du projet.



(le petit nombre a droite est égale la date de debut de relisation du la tache précédent - duré la tache, sois la date maximal pour commencer la tache)

Chap 7 (flots)

Flot Compatible

Un flot compatible c'est un flot qui respect l'équation de conservation

Algorithme de Ford et Fulkerson

Pas polinomial, du au fait que suivant les choix de recherche, on peut avoir + ou - d'itération

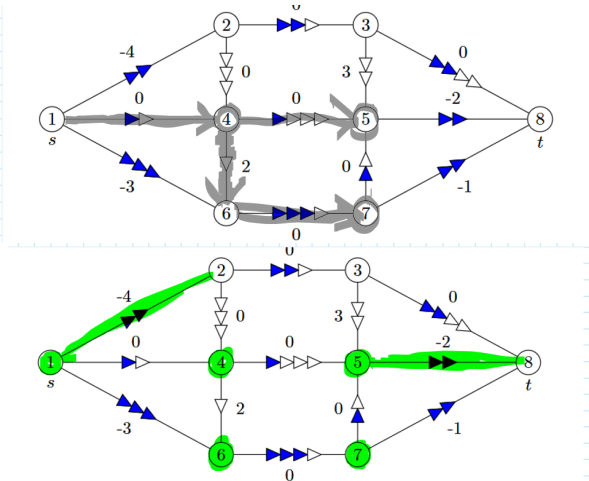
1. prendre un flot initial possible
2. Crée R^* (réseau d'augmentation)
3. chercher le chemin de s à t dans R^*
4. si trouver, augmenter au maximum le flot le long de ce chemin (en deminant les flot inverse)
5. répéter jusqu'à qu'on ai plus de chemin dans R^*

Coupe

En gros, l'idée c'est de séparé les sommet en 1 group A, et de faire en sort que chaque arc du group A vas en dehors de celui ci

La capacité de cette coupe est defini par la comme des capacité des arc formant la coupe

si notre flow est égale a la capacité de al coupe, alors la coupe est minimal et le flot est maximal
pour le trouver,



Théorème du flot max et de la coupe min

Dans un réseau, la valeur du flot max de S à T est toujours égale à la capacité min d'une coupe séparant s de t

Couplage maximum dans un graphe biparti

Dans un cas où le graph est bipartie, on peut voir le problème comme une résolution de couplage max dans un graphe bipartie.

en gros, on prend notre graph :

- on oriente tous les arcs d'un groupe vers l'autre
- on met la capacité de chaque arc à 1
- on ajoute une source "s" et un pui "t"
- on cherche le flot max de s à t

Problème à flot max à coût (\$) min

en gros, on met un coup de n par unité sur les arcs, dans ce réseau on cherche le flot max mais à coût min

Algorithme de Busacker et Gowenver

1. on commence avec un flot initial nul
2. Construire un réseau d'augmentation R*
3. Cherche un plus court chemin de s à t dans R*
 - si trouver, répéter depuis (2)
 - (si coût) mettre à jour les cout en faisant $c_{ij} = c_{ij} + (\lambda_i - \lambda_j)$ (pour éviter les cout négatifs)
4. end

Problème d'affectation linéaire et transbordement

on a N personnes avec N tâches et on cherche comment les répartir de manière qu'on ait une tâche par personne.

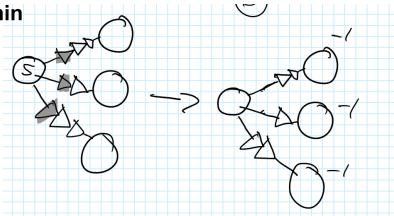
En gros, c'est problème de couplage parfait de coût minimum dans un graphe bipartie.

donc on a :

- des sources avec N unités
- des puits avec qui doivent demander le même nombre d'unités
- et des sommets de transit avec ni offre ni demande

BTW, un pui c'est pas seulement un sommet sans successeur, mais bien à partir du moment où un sommet consomme quelque chose

Éliminer un flow à capacité min

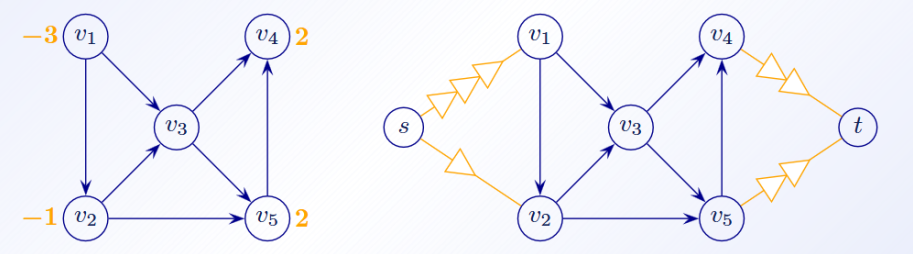


Définition d'un problème

- donner ce que l'on cherche
- définir les différentes parties de notre modélisation
- définir comment savoir si on a résolu le problème

Other shit

- on a un réseau R avec une *func* associant chaque sommet à une capacité
 - splitter chaque sommets en 2 (sortant et entrant), mettre l'arc entre entrant et sortant à la capacité *func*, résoudre le problème de flot
- Graph G, trouver le nombre max de chemins de s à t disjoint par
 - arc, on met une capacité de 1 par arc into flow max
 - sommet, on met une capacité de 1 par sommet into flow max en utilisant la technique ci-dessus
- trouver le plus court chemin dans un graph à pondération
 - arc de s et de t ont une capacité de 1, les autres arcs auront aussi une capacité de 1 et aussi le même coût unitaire que la pondération, into flowmax
- trouver arbre min
 - Source s avec une capacité de |V|-1 et tous les autres sommets ont une demande 1 puis même logique



Couplage

- Parfait = chaque Edge est couplée avec une autre
- Maximum = en gros, c'est quand on utilise la maximum d'arc et que l'on ne peut plus en rajouter

Chain

- Chaîne alternée relative à M : chaîne dans l'alternance classique



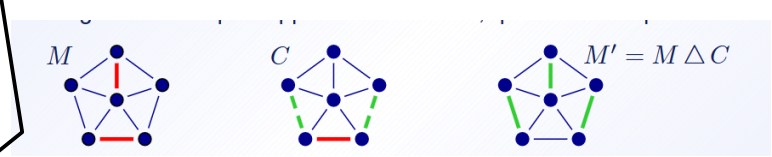
- Chaîne alternée augmentante à M : si y a les deux extrémités de la chaîne qui ne sont pas dans M



Théorème de Berge

en gros, il dit que si on a une chaîne donnée C, on peut lui ajouter un membre en faisant :

- crée un couplage d'arc non compris dans C
- faire $C \Delta M$



Recherche de couplage maximum

1. crée un couplage, peu importe lequel
2. Oriente les arcs :
 - droite → gauche
 - et gauche → droite
3. en partant du sommet "source" et allant vers un "puits" on explore une chaîne alternée C augmentante sur M
4. on fait $C \Delta M$

Other shit

- graph bipartie $K_{r,s}$ (r + s sommets)
 - $nArc = r * s$ / couplage max cardinalité = $\min(r, s)$
- Si on a le couplage max d'un graph G, trouver le recouvrement → ajoute les sommets manquants pour former l'arbre
- Si on a un arbre recouvrant min, trouver le couplage max = garder un arc pour tous les sommets à plus de 1 connexion

Graph bipartie complet

c'est un graph bipartie dans lequel chaque Edge d'un group est relié à tous les autres Edges de l'autre group

- Recouvrement = sous-ensemble d'arcs tel que chaque sommet de G soit au moins un extrémité des arcs
- transversal = sous-ensemble d'arcs

